

TP FIL ROUGE — DOMAIN-DRIVEN DESIGN

Billetterie & Réservation

Soheil BENABIDA, Lucas PIRES

7 Bounded Contexts

7 Invariants — 22 tests

Architecture hexagonale

Contexte métier

Plateforme de billetterie en ligne **multi-salles** (concerts, théâtre, conférences). **Forte saisonnalité** et pics de demande à l'ouverture des ventes. Contraintes de capacité physique strictes.

Concurrence d'accès

Centaines de clients simultanés sur les mêmes places.
Verrou temps réel — unicité garantie.

Tarification complexe

Tarifs de base + réductions (abonné, étudiant, senior) + prévente.
Règles métier encapsulées dans un contexte dédié.

Fraude & intégrité

Bots, revente non autorisée, billets dupliqués.
QR code unique vérifié à l'entrée.

7 Bounded Contexts identifiés par l'analyse du domaine

ContexteRéservation

ContexteTarification

ContexteProgrammation

ContexteContrôleAccès

ContexteCompteClient

ContextePaiement

ContexteNotification

Classification DDD des sous-domaines

CORE

Réservation

Cœur de valeur
Verrou temps réel
Concurrence d'accès

Tarification

Avantage concurrentiel
Règles tarifaires
Complexité métier

SUPPORTING

Programmation

Événements & salles
Données de référence
Périodes de vente

Contrôle Accès

QR validation
Détection fraude
Registre billets

Compte Client

Profils & abonnements
Historique réservations
Source réductions

GENERIC

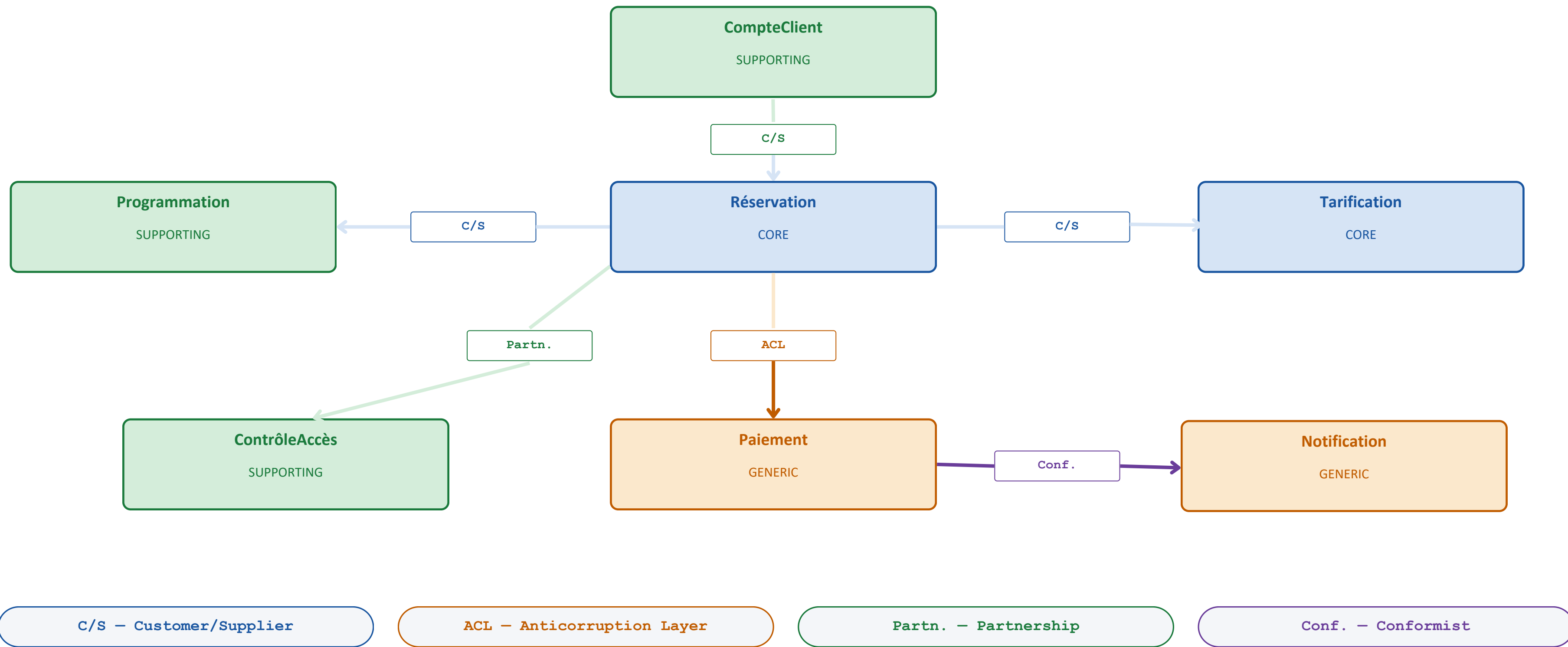
Paieement

Délégué à Stripe
Webhook + ACL
Remboursements

Notification

SendGrid · Twilio
Email / SMS / push
Conformiste

Context Map & Patterns d'intégration



Agrégats clés — Réservation & Événement

«Agrégat» Réservation (Core · 4 INV)

«Agrégat» Réservation

— «Racine» Réservation

réservationId · statut · date

— «OV» MontantTarifé [1]

tarifBase · réduction · total

— «Entité» Billet [1..10]

billetId · qrCode · statut

— «OV» PlaceAssignée

«Agrégat» Événement (Supporting · 3 INV)

«Agrégat» Événement

— «Racine» Événement

événementId · titre · jauge · statut

— «OV» PériodeDeVente [1]

débutPrévente · finVente

— «OV» Place [1..*]

zone · rangée · numéro

7 invariants gardés par les agrégats

INV-R1	1 à 10 places par réservation — prévient la revente
INV-R2	Unicité des places — pas de double attribution
INV-R3	Paiement exact requis — Confirmée impossible sans PaiementAccepté
INV-R4	Statuts terminaux irréversibles — Annulée/Remboursée : état final

INV-E1	Jauge ≥ 0 — sur-réservation physiquement impossible
INV-E2	Complet bloque les ventes — jauge = 0 → aucune réservation
INV-E3	Période de vente respectée — fenêtre ouverte obligatoire

Fil rouge — Mohammed réserve 2 places

Mohamed Dupont (CLI-2025-00089, abonnée) — 2 places Orchestre A12+A13 — Concert Jazz EVT-2025-00034 — Grande Salle

1	Vérification dispo	[Programmation]	23 places restantes · ventes ouvertes (INV-E1, INV-E2, INV-E3)
2	Calcul tarif	[Tarification]	90 € × 2 – 20 % abonnée – 18 € + 4 € frais → 76 €
3	RéservationCréée	[Réservation]	RES-2025-00142 EnAttente · jauge 23 → 21 · INV-R1, INV-R2 ✓
4	Paie ment Stripe	[Paie ment (ACL)]	Transaction transmise via ACL · webhook Stripe attendu
5	Paie mentAccepté	[ACL → Réservation]	76 € = montant réservation · INV-R3 ✓ · statut → Confirmée
6	RéservationConfirmée	[Réservation]	BIL-00283 (QR-X7K9M) + BIL-00284 (QR-P3L2N) générés
7	Email + billets PDF	[Notification]	SendGrid · envoi en < 30 s · 2 billets en pièce jointe
8	Scan à l'entrée	[ContrôleAccès]	QR-X7K9M validé → statut Utilisé · puis QR-P3L2N · Accès ✓

INV-R1 ✓

INV-R2 ✓

INV-R3 ✓

INV-E1 ✓

INV-E2 ✓

INV-E3 ✓

Architecture hexagonale

Le **Domain ne dépend de rien**. Il définit les **ports** (interfaces). Les Adapters les implémentent — toute la technique est en périphérie.

ADAPTERS — Entrants	APPLICATION — Use Cases	DOMAIN — Cœur métier
Contrôleur REST · Consommateur RabbitMQ · CLI POST /api/reservations POST /api/reservations/{id}/confirmer → CréerRéservationCommande	CréerRéservation · ConfirmerPaie^{ment} · AnnulerRéservation Coordonne Domain + Adapters Gère les transactions Publie les événements de domaine	Réservation · Billet · Événement · Salle MontantTarifé · PlaceAssignée Ports (interfaces) · Zéro dépendance Invariants vérifiés : INV-R1/R2/R3/R4 INV-E1/E2/E3 Aucune import technique

ADAPTERS SORTANTS → PostgreSQL Repository · RabbitMQ Publisher · Stripe (ACL) · SendGrid

Flux — POST /api/reservations

```
REST Controller → CréerRéservationCommande → UseCase
├─> ÉvénementRepository (vérif. jauge + période)
├─> ServiceTarification (76 €) · Domaine : INV-R1/R2 ✓
└─> RéservationRepository.save() · EventBus.publish(RéservationCréée) → HTTP 201
```

Architecture Runtime — Docker Compose

1 conteneur par Bounded Context — déploiement indépendant, scalabilité ciblée.

api-reservation :8080 · ContexteRéservation Core — dépend de postgres, rabbitmq, tarification	api-tarification :8081 · ContexteTarification Stateless — calcul de prix à la demande	api-notification :8082 · ContexteNotification Subscriber RabbitMQ → SendGrid · scalable	api-contrôle-acces :8083 · ContexteContrôleAccès Subscriber + registre QR · on-site ou cloud
postgres :5432 · Infrastructure PostgreSQL 16-alpine — réservations, billets, QR	rabbitmq :5672 · Message Broker RabbitMQ 3.13 — AMQP + UI management (:15672)	prometheus :9090 · Observabilité Scraping /metrics toutes les 15 s	grafana :3000 · Observabilité Dashboards + alertes Slack / PagerDuty

Sécurité : utilisateur non-root · python:3.12-slim · secrets via CI .env jamais versionnés | **healthcheck** sur /metrics avant trafic

Observabilité — 3 piliers

① Correlation ID

UUID généré à l'entrée (si absent).
Propagé en header HTTP et dans
chaque message RabbitMQ.
Relie tous les logs d'un même flux.

```
def _correlation_id(self) -> str:
    return (
        self.headers.get("X-Correlation-ID")
        or f"COR-{uuid.uuid4().hex[:12].upper()}"
    )
```

② Logging structuré

Format JSON, un objet par ligne.
Exploitable par ELK, Loki, etc.
4 champs clés :
timestamp · level · logger · correlation_id.

```
{
  "level":          "INFO",
  "message":        "RéservationConfirmée",
  "correlation_id": "COR-XYZ",
  "context":        "ContexteNotification"
}
```

③ Métriques /metrics

Endpoint JSON scrapé par Prometheus
toutes les 15 secondes.
2 Counters + 1 Gauge.
Affiché dans Grafana.

```
{
  "reservations_creees_total": 42,
  "http_requests_total":      127,
  "jauge_disponible": {
    "EVT-2025-00034": 21
  }
}
```

3 métriques exposées sur GET /metrics

MÉTIER

reservations_creees_total
Counter — volume d'activité, pics à l'ouverture

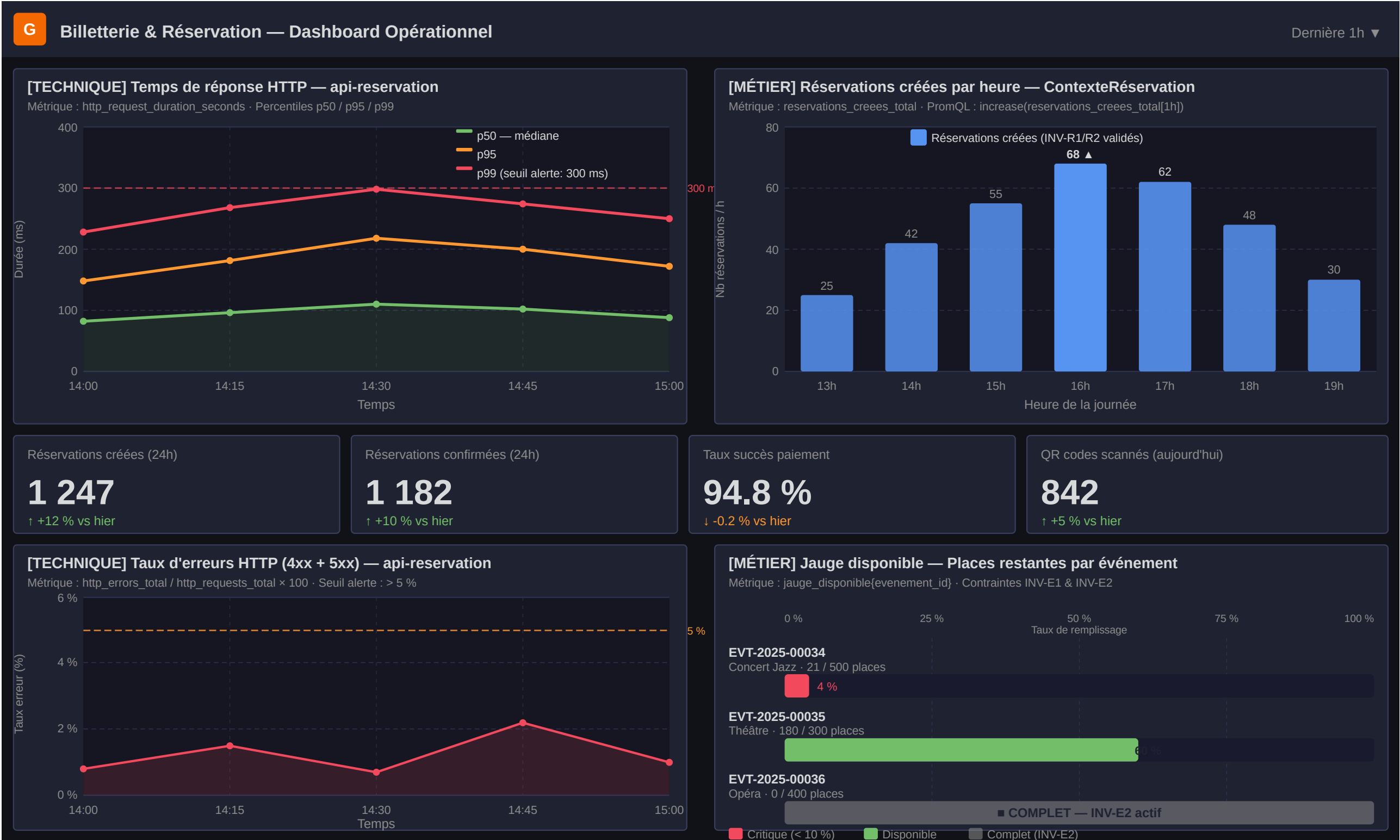
TECHNIQUE

http_requests_total
Counter — santé du service REST

LIBRE

jauge_disponible{event}
Gauge — places restantes par événement

Dashboard Grafana — Mockup opérationnel



Synthèse — Ce que DDD nous a apporté

Bounded Contexts

7

2 Core · 3 Supporting · 2 Generic

Invariants

7

4 Réservation · 3 Événement

Tests passants

22/22

Couverture complète des invariants

Patterns DDD

4

C/S · ACL · Partnership · Conformist

Alignement métier ↔ code

L'agrégat "Réservation" dans le code correspond exactement au terme métier. Le langage ubiquitaire élimine la traduction — réduction du risque d'incompréhension.

Robustesse par invariants

Les 7 règles métier sont gardées par les agrégats, pas par des validations éparpillées. INV-R1, INV-E2... testés, documentés, inviolables.

Découplage explicite par contextes

7 contextes évoluent indépendamment. Stripe est remplaçable sans toucher au domaine. Chaque pattern de relation est documenté et justifié.

Observabilité orientée métier

Les métriques sont alignées sur les événements de domaine : réservations créées, jauge disponible, taux de confirmation — pas seulement les métriques techniques.

FIN DE LA PRÉSENTATION

Merci.

Questions & échanges

`docs/` — 21 fichiers Markdown

`tests/integration/` — 22 tests

`presentation.pptx`